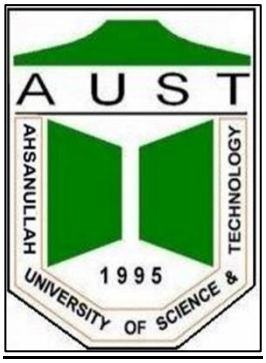
AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2104
Course Title: Data Structures Lab

Assignment No: 03

Date of Submission: 15/07/2026

Submitted by,

Name: Hasibul Hasan
Student ID: 00724205101098
Lab Section: B2
Academic Semester: Fall 2025
Program: BSc in CSE

CSE2104 Offline-3 Report

This report contains the solutions, source code, and sample runs for the 10 tasks in Data Structure Lab (CSE2104) Offline Assignment 3. The first 6 tasks cover quick sort implementations, variations, and metrics. Tasks 7 to 10 cover merge sort applications, comparisons count, merge operations, and recursion calls.

Task 1: Quick Sort Comparison Count

Problem Statement: Given an array of N integers, simulate Quick Sort using the last element as pivot and count the number of comparisons performed during the sorting process.

Source Code (C++):

```
/* 1. Given an array of N integers, simulate Quick Sort using the last
element as pivot and
count the number of comparisons performed during the sorting process.
*/

#include<bits/stdc++.h>
using namespace std;
int comp=0;
int Sort(vector<int>&v,int high,int low)
{
    int pv=v[high];
    int i=low;
    int j=high;

    while(i<j){
     while(i<high && pv>v[i])
     {
        i++;
        comp++;
     }
    comp++;
     while(j>low && pv<=v[j])
     {
        j--;
        comp++;
        }
    comp++;
    if(i<j)
    {
     swap(v[i],v[j]);
    }


    }
```

```
    swap(v[i],v[high]);
    return i;
}
void quickSort(vector<int>&v,int high,int low)
{

    if(high>low)
    {
        int pt=Sort(v,high,low);
        quickSort(v,pt-1,low);
        quickSort(v,high,pt+1);
    }
}

int main()
{
    int n;
vector<int>v;
cin>>n;
for(int i=0;i<n;i++)
{
     int x;
     cin>>x;
     v.push_back(x);

}

quickSort(v,n-1,0);
for(auto x:v)
{
     cout<<x<<" ";
}
cout<<endl<<comp;

}
```

Sample Input	Sample Output
6 12 7 15 5 10 8	5 7 8 10 12 15 22

Task 2: Partitioning Array around Pivot

Problem Statement: Given an array and a pivot, rearrange the array such that elements less than pivot come before it and greater come after.

Source Code (C++):

```
/* 2. Given an array and a pivot, rearrange the array such that
elements less than pivot come
before it and greater come after.*/
#include<bits/stdc++.h>
using namespace std;

void Sort(vector<int>&v,int high,int low,int pv)
{

    int i=low;
    int j=high;
    int k;
for(k=low;k<=high;k++)
{
    if(v[k]==pv)
    {
        break;
    }
}
swap(v[k],v[high]);
    while(i<j){
     while(i<high && pv>v[i])
     {
        i++;

     }

     while(j>low && pv<=v[j])
     {
        j--;

        }

    if(i<j)
    {
     swap(v[i],v[j]);
    }


    }

    swap(v[i],v[high]);

}
```

```
void quickSort(vector<int>&v,int high,int low,int pv)
{

    if(high>low)
    {
        Sort(v,high,low,pv);

    }
}

int main()
{
    int n;
vector<int>v;
cin>>n;
for(int i=0;i<n;i++)
{
     int x;
     cin>>x;
     v.push_back(x);

}
int pv;
cin>>pv;
quickSort(v,n-1,0,pv);
for(auto x:v)
{
     cout<<x<<" ";
}


}
```

Sample Input	Sample Output
6 12 7 15 5 10 8 10	8 7 5 10 12 15

Task 3: Quick Sort Swap Count

Problem Statement: Count the number of swaps required to sort an array using quick sort.

Source Code (C++):

```
//3. Count the number of swaps required to sort an array using quick
sort.

#include<bits/stdc++.h>
using namespace std;
int Count=0;
int Sort(vector<int>&v,int high,int low)
{
    int pv=v[high];
    int i=low;
    int j=high;

    while(i<j){
     while(i<high && pv>v[i])
     {
        i++;

     }

     while(j>low && pv<=v[j])
     {
        j--;

        }

    if(i<j)
    {
        Count++;
     swap(v[i],v[j]);
    }


    }

    swap(v[i],v[high]);
    Count++;
    return i;
}
void quickSort(vector<int>&v,int high,int low)
{

    if(high>low)
    {
        int pt=Sort(v,high,low);
        quickSort(v,pt-1,low);
```

```
        quickSort(v,high,pt+1);
    }
}

int main()
{
    int n;
vector<int>v;
cin>>n;
for(int i=0;i<n;i++)
{
     int x;
     cin>>x;
     v.push_back(x);

}

quickSort(v,n-1,0);
for(auto x:v)
{
     cout<<x<<" ";
}
cout<<endl<<Count;

}
```

Sample Input	Sample Output
6 12 7 15 5 10 8	5 7 8 10 12 15 5

Task 4: Output Chosen Pivot at Each Recursion

Problem Statement: Output the pivot chosen at each recursive call in quick sort (always pick the last element as pivot).

Source Code (C++):

```
/* Output the pivot chosen at each recursive call in quick sort (always
pick the last element as pivot).*/

#include<bits/stdc++.h>
using namespace std;

int Sort(vector<int>&v,int high,int low)
{
    int pv=v[high];
    int i=low;
    int j=high;
cout << "Pivot: " << v[high] << endl;
    while(i<j){
     while(i<high && pv>v[i])
     {
        i++;

     }

     while(j>low && pv<=v[j])
     {
        j--;

        }

    if(i<j)
    {

     swap(v[i],v[j]);
    }


    }

    swap(v[i],v[high]);

    return i;
}
void quickSort(vector<int>&v,int high,int low)
{

    if(high>low)
    {
        int pt=Sort(v,high,low);
```

```
        quickSort(v,pt-1,low);
        quickSort(v,high,pt+1);
    }
}

int main()
{
    int n;
vector<int>v;
cin>>n;
for(int i=0;i<n;i++)
{
     int x;
     cin>>x;
     v.push_back(x);

}

quickSort(v,n-1,0);
for(auto x:v)
{
     cout<<x<<" ";
}


}
```

Sample Input	Sample Output
6 12 7 15 5 10 8	Pivot: 8 Pivot: 7 Pivot: 15 Pivot: 10 5 7 8 10 12 15

Task 5: Output Pivot Placement Index

Problem Statement: For each recursive quick sort call, output the final index where pivot is placed.

Source Code (C++):

```
/* 5.For each recursive quick sort call, output the final index where
pivot is placed. */
#include<bits/stdc++.h>
using namespace std;

int Sort(vector<int>&v,int high,int low)
{
    int pv=v[high];
    int i=low;
    int j=high;

    while(i<j){
     while(i<high && pv>v[i])
     {
        i++;

     }

     while(j>low && pv<=v[j])
     {

        j--;

        }

    if(i<j)
    {

     swap(v[i],v[j]);
    }


    }

    swap(v[i],v[high]);
        cout << "Pivot Index: " << i<< endl;
    return i;
}
void quickSort(vector<int>&v,int high,int low)
{

    if(high>low)
    {
        int pt=Sort(v,high,low);
```

```
        quickSort(v,pt-1,low);
        quickSort(v,high,pt+1);
    }
}

int main()
{
    int n;
vector<int>v;
cin>>n;
for(int i=0;i<n;i++)
{
     int x;
     cin>>x;
     v.push_back(x);

}

quickSort(v,n-1,0);
for(auto x:v)
{
     cout<<x<<" ";
}


}
```

Sample Input	Sample Output
6 12 7 15 5 10 8	Pivot Index: 2 Pivot Index: 1 Pivot Index: 5 Pivot Index: 3 5 7 8 10 12 15

Task 6: Count Elements Less Than Pivot

Problem Statement: For each pivot selected in quick sort (last element as pivot), count how many elements are less than the pivot at that step.

Source Code (C++):

```
/* 6.For each pivot selected in quick sort (last element as pivot),
count how many elements are less than the pivot at that step.*/
#include<bits/stdc++.h>
using namespace std;

int Sort(vector<int>&v,int high,int low)
{
    int pv=v[high];
    int i=low;
    int j=high;

    while(i<j){
     while(i<high && pv>v[i])
     {
        i++;

     }

     while(j>low && pv<=v[j])
     {

        j--;

        }

    if(i<j)
    {

     swap(v[i],v[j]);
    }


    }

    swap(v[i],v[high]);
    cout << "Element less than pivot "<<v[i] << ": "<< i << endl;
    return i;
}
void quickSort(vector<int>&v,int high,int low)
{

    if(high>low)
    {
        int pt=Sort(v,high,low);
```

```
        quickSort(v,pt-1,low);
        quickSort(v,high,pt+1);
    }
}

int main()
{
    int n;
vector<int>v;
cin>>n;
for(int i=0;i<n;i++)
{
     int x;
     cin>>x;
     v.push_back(x);

}

quickSort(v,n-1,0);
for(auto x:v)
{
     cout<<x<<" ";
}


}
```

Sample Input	Sample Output
6 12 7 15 5 10 8	Element less than pivot 8: 2 Element less than pivot 7: 1 Element less than pivot 15: 5 Element less than pivot 10: 3 5 7 8 10 12 15

Task 7: Merge Sort Combined Median

Problem Statement: Given two sorted arrays, find the median of the combined sorted array using merge sort. Also print the merged array.

Source Code (C++):

```
/* 7. Given two sorted arrays, find the median of the combined sorted
array using merge sort.
Also print the merged array.*/

#include<bits/stdc++.h>
using namespace std;

void Merge(vector<double> &v, int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = low;

    vector<double> temp(v.size());

    while(i <= mid && j <= high)
    {
        if(v[i] <= v[j])
        {
            temp[k] = v[i];
            i++;
        }
        else
        {
            temp[k] = v[j];
            j++;
        }
        k++;
    }

    while(i <= mid)
    {
        temp[k] = v[i];
        i++;
        k++;
    }

    while(j <= high)
    {
        temp[k] = v[j];
        j++;
        k++;
    }
```

```
    for(int x = low; x <= high; x++)
        v[x] = temp[x];
}

void merge_sort(vector<double> &v, int low, int mid, int high)
{
    Merge(v, low, mid, high);
}

int main()
{
    int n1, n2;
    cin >> n1 >> n2;

    vector<double> a(n1), b(n2);

    for(auto &x : a)
        cin >> x;

    for(auto &x : b)
        cin >> x;

    vector<double> v;

    for(auto &x : a)
        v.push_back(x);

    for(auto &x : b)
        v.push_back(x);

    merge_sort(v, 0, n1 - 1, n1 + n2 - 1);

    cout << "Merged Array: ";
    for(auto x : v)
        cout << x << " ";

    double med;

    int n = n1 + n2;

    if(n % 2 == 1)
    {
        med = v[n / 2];
    }
    else
        med = (v[n / 2 - 1] + v[n / 2]) / 2.0;

    cout << "\nMedian: " << med;

    return 0;
}
```

Sample Input	Sample Output
4 3 1 3 5 7 2 4 6	Merged Array: 1 2 3 4 5 6 7 Median: 4

Task 8: Merge Sort Comparison Count

Problem Statement: Calculate the total number of comparisons made during merge sort.

Source Code (C++):

```
// 8. Calculate the total number of comparisons made during merge sort.

#include<bits/stdc++.h>
using namespace std;
int comp=0;
void merge(vector<int>&v,int low,int mid,int high)
{
    int i=low;
    int j=mid+1;
    int k=low;
    vector<int>temp;
    while(i<=mid && j<=high)
    {

        if(v[i]<=v[j])
        {
            temp.push_back(v[i]);
            i++;
            k++;
            comp++;
        }
        else{
            temp.push_back(v[j]);
            j++;
            k++;
        }
        comp++;
    }
    if(i>mid)
    {
        while(j<=high)
        {
            temp.push_back(v[j]);
            j++;
            k++;
        }
    }
    else{
        while(i<=mid)
    {
        temp.push_back(v[i]);
            i++;
            k++;
    }
    }
```

```
    int g=0;
    for(int i=low;i<=high;i++)
    {
        v[i]=temp[g++];
    }

}
void mergeSort(vector<int>&v,int low,int high)
{
if(high>low)
{
int mid=(low+high)/2;
    mergeSort(v,low,mid);
    mergeSort(v,mid+1,high);

merge(v,low,mid,high);

}

}


int main()
{
int n;
    cin>>n;
        vector<int>v;
        for(int i=0;i<n;i++)
        {
            int x;
            cin>>x;
            v.push_back(x);
        }
        mergeSort(v,0,n-1);
        for(auto y:v)
        {
            cout<<y<<" ";
        }
    cout<<endl<<comp;
}
```

Sample Input	Sample Output
6 12 7 15 5 10 8	5 7 8 10 12 15 15

Task 9: Count Merge Operations

Problem Statement: Count the total number of merge operations performed when sorting an array using merge sort.

Source Code (C++):

```
/* 9. Count the total number of merge operations performed when sorting
an array using merge
sort. */

#include<bits/stdc++.h>
using namespace std;
int cnt=0;
void merge(vector<int>&v,int low,int mid,int high)
{
    cnt++;
    int i=low;
    int j=mid+1;
    int k=low;
    vector<int>temp;
    while(i<=mid && j<=high)
    {
        if(v[i]<=v[j])
        {
            temp.push_back(v[i]);
            i++;
            k++;
        }
        else{
            temp.push_back(v[j]);
            j++;
            k++;
        }
    }
    if(i>mid)
    {
        while(j<=high)
        {
            temp.push_back(v[j]);
            j++;
            k++;
        }
    }
    else{
        while(i<=mid)
    {
        temp.push_back(v[i]);
            i++;
            k++;
    }
```

```
    }
    int g=0;
    for(int i=low;i<=high;i++)
    {
        v[i]=temp[g++];
    }

}
void mergeSort(vector<int>&v,int low,int high)
{
if(high>low)
{
int mid=(low+high)/2;
    mergeSort(v,low,mid);
    mergeSort(v,mid+1,high);

merge(v,low,mid,high);

}

}


int main()
{
int n;
    cin>>n;
        vector<int>v;
        for(int i=0;i<n;i++)
        {
            int x;
            cin>>x;
            v.push_back(x);
        }
        mergeSort(v,0,n-1);
        for(auto y:v)
        {
            cout<<y<<" ";
        }
        cout<<endl<<cnt;

}
```

Sample Input	Sample Output
6 12 7 15 5 10 8	5 7 8 10 12 15 5

Task 10: Count Merge Sort Recursive Calls

Problem Statement: Determine the total number of recursive calls merge sort makes for a given array.

Source Code (C++):

```
/* 10. Determine the total number of recursive calls merge sort makes
for a given array. */

#include<bits/stdc++.h>
using namespace std;
int cnt=0;
void merge(vector<int>&v,int low,int mid,int high)
{
    int i=low;
    int j=mid+1;
    int k=low;
    vector<int>temp;
    while(i<=mid && j<=high)
    {
        if(v[i]<=v[j])
        {
            temp.push_back(v[i]);
            i++;
            k++;
        }
        else{
            temp.push_back(v[j]);
            j++;
            k++;
        }
    }
    if(i>mid)
    {
        while(j<=high)
        {
            temp.push_back(v[j]);
            j++;
            k++;
        }
    }
    else{
        while(i<=mid)
    {
        temp.push_back(v[i]);
            i++;
            k++;
    }
    }
    int g=0;
```

```
    for(int i=low;i<=high;i++)
    {
        v[i]=temp[g++];
    }

}
void mergeSort(vector<int>&v,int low,int high)
{
    cnt++;
if(high>low)
{
int mid=(low+high)/2;
    mergeSort(v,low,mid);
    mergeSort(v,mid+1,high);

merge(v,low,mid,high);

}

}


int main()
{
int n;
    cin>>n;
        vector<int>v;
        for(int i=0;i<n;i++)
        {
            int x;
            cin>>x;
            v.push_back(x);
        }
        mergeSort(v,0,n-1);
        for(auto y:v)
        {
            cout<<y<<" ";
        }
        cout<<endl<<cnt;

}
```

Sample Input	Sample Output
6 12 7 15 5 10 8	5 7 8 10 12 15 11